



Software Testing — Complete Notes

Quality concepts · Unit, integration, E2E, TDD, BDD & more

A reader-friendly, all-in-one reference for software testing concepts.

1. 🎯 Why Testing

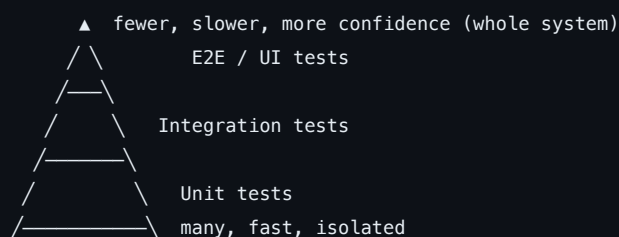
Software testing is the practice of **checking that code behaves as intended** — automatically and repeatably. Good tests catch bugs early (when they're cheap to fix), document expected behavior, enable confident refactoring, and are the safety net that makes **CI/CD** possible.

💡 **Mental model:** tests are an **executable specification**. They say "given this input, expect this output" and fail loudly when reality drifts. The earlier a bug is caught, the cheaper it is — a unit test catches it in seconds; production catches it in an incident.

Goals: correctness, prevent regressions, enable safe change, document behavior, and give fast feedback.

2. ▲ The Testing Pyramid

A guideline for **how many** of each test type to write: many fast low-level tests, few slow high-level ones.



Level	Speed	Scope	How many
Unit	ms	One function/class in isolation	Most (broad base)
Integration	seconds	Modules together (DB, API)	Some (middle)
E2E	minutes	Whole app, user flows	Few (thin top)

⚠️ Anti-patterns: the **ice-cream cone** (too many slow E2E, few units) and the **hourglass** (lots of unit + E2E, no integration).
Aim for a wide, fast base.

3. 🖋️ Unit Testing

- Tests **one small unit** (a function/method/class) **in isolation** from its dependencies.
- **Fast, deterministic, numerous** — run on every save/commit.
- External dependencies (DB, network, time) are replaced with **test doubles** (mocks/stubs).
- Structure: **Arrange** → **Act** → **Assert** (AAA), or Given-When-Then.

```
# Arrange-Act-Assert
def test_add():
    calc = Calculator()           # arrange
    result = calc.add(2, 3)       # act
    assert result == 5            # assert
```

Good unit tests are **FIRST**: Fast, Isolated, Repeatable, Self-validating, Timely.

4. 🔗 Integration Testing

- Verifies that **multiple components work together** — e.g. service ↔ database, API ↔ external service, module ↔ module.
- Catches bugs unit tests can't: wrong SQL, serialization mismatches, wiring/config errors, contract mismatches.
- Slower than unit tests (real DB/queue, often via **containers** like Testcontainers); fewer of them.
- May use a **real test database** or in-memory substitute.

The line: a **unit test** mocks the database; an **integration test** uses a real (test) database to verify the queries actually work.

5. 🌐 End-to-End (E2E) Testing

- Tests the **entire application from the user's perspective**, through the real UI/API, with real (or production-like) dependencies.
- Validates complete **user journeys** (sign up → add to cart → checkout).
- **Slowest, most brittle, most realistic** — keep few, cover critical paths only.
- Tools drive a real browser (Cypress, Playwright, Selenium).

E2E gives the highest confidence but the worst feedback loop and maintenance cost — reserve it for the handful of flows that absolutely must work.

6. Other Test Types

Type	Checks
Smoke	Does the build basically work? (quick sanity before deeper tests)
Regression	Did a change break previously working features?
Acceptance (UAT)	Does it meet business/user requirements?
Performance / Load / Stress	Speed, throughput, behavior under heavy/extreme load
Security	Vulnerabilities (SAST/DAST, pen testing)
Smoke vs Sanity	Smoke = broad shallow build check; Sanity = narrow deep check of a specific fix
Exploratory	Unscripted human investigation
Accessibility (a11y)	Usable by people with disabilities
Snapshot	Output matches a saved reference (UI components)

7. Functional vs Non-Functional

	Functional	Non-Functional
Asks	Does it do what it should?	How well does it do it?
Examples	Login works, cart totals correctly	Speed, scalability, security, usability, reliability
Types	Unit, integration, E2E, acceptance	Performance, load, stress, security, accessibility

8. TDD (Test-Driven Development)

Write the **test first**, then the code to pass it. The cycle is **Red** → **Green** → **Refactor**:

```
▼
┌ RED: write a failing test for new behavior |
├ |                                         |
├ GREEN: write the minimum code to pass    |
├ |                                         |
└ REFACTOR: clean up, tests still green —┘
```

1. **Red** — write a small failing test for the next bit of behavior.
2. **Green** — write the simplest code that makes it pass.
3. **Refactor** — improve the design while keeping tests green.

Benefits: forces testable design, gives instant feedback, builds a safety net, prevents over-engineering (you only write code a test demands). **Cost:** discipline + slower upfront. TDD is a *design* technique as much as a testing one.

9. 🥦 BDD (Behavior-Driven Development)

An evolution of TDD that describes behavior in **plain, business-readable language** so devs, QA, and product share one spec.

- Scenarios use **Given–When–Then** (Gherkin syntax):

```
Feature: Checkout
  Scenario: Successful purchase
    Given a customer with an item in the cart
    When they complete the checkout
    Then the order is confirmed
    And a receipt email is sent
```

- Tools (**Cucumber**, **SpecFlow**, **Behave**) map each Gherkin step to code.
- **TDD vs BDD:** TDD tests *units* in code terms ("function returns 5"). BDD tests *behavior* in business terms ("order is confirmed") — improves collaboration and living documentation.

10. 🧭 Other Approaches (ATDD, etc.)

Approach	Idea
ATDD (Acceptance Test-Driven)	Define acceptance criteria as tests <i>before</i> building; whole team agrees on "done".
Contract testing	Verify the agreement between a service and its consumers (e.g. Pact) — catches breaking API changes.
Property-based testing	Assert properties over many auto-generated inputs (Hypothesis, QuickCheck) instead of fixed examples.
Mutation testing	Introduce bugs ("mutants") to check your tests actually catch them — measures test <i>quality</i> .
Fuzz testing	Feed random/malformed input to find crashes/security holes.
Snapshot testing	Compare output against a stored baseline.

11. 🤖 Test Doubles (Mocks, Stubs...)

"Test double" = any stand-in for a real dependency. Types (by Gerard Meszaros):

Double	Purpose
Dummy	Passed but never used (fills a parameter).
Stub	Returns canned answers to calls.
Spy	A stub that also records how it was called.
Mock	Pre-programmed with expectations ; the test <i>verifies</i> it was called correctly.
Fake	A working lightweight implementation (e.g. in-memory DB).

Rule of thumb: **stub** to control inputs ("when called, return X"); **mock** to assert interactions ("expect this was called once with Y"). Don't over-mock — it couples tests to implementation.

12. 🇮🇹 Coverage & Quality Metrics

- **Code coverage** = % of code exercised by tests (line, branch, statement, function).
- High coverage ≠ good tests — it shows what ran, **not** that assertions are meaningful. 100% coverage with weak asserts is false confidence.
- **Mutation score** is a better quality signal (do tests catch injected bugs?).
- Aim for **meaningful coverage of critical paths**, not a vanity number; watch **flaky tests** (pass/fail nondeterministically) — they erode trust and must be fixed or quarantined.

13. 🔄 Testing in CI/CD

- Tests are the **gate** in a pipeline — a merge/deploy only proceeds if they pass. See the [CI/CD notes](#).
- **Fail fast**: run cheap, broad tests first (lint → unit), slow/narrow ones later (integration → E2E).
- Run unit tests on **every push/PR**; integration in CI with **ephemeral containers**; E2E on a deployed staging environment.
- **Shift left** — test as early as possible; publish results + coverage as build artifacts; block merges on red.

14. 🛠️ Tools by Language

Language / area	Common tools
JavaScript/TS	Jest, Vitest, Mocha, Playwright, Cypress, Testing Library
Python	pytest, unittest, Hypothesis, tox
Java	JUnit, TestNG, Mockito, REST Assured
Go	built-in <code>testing</code> , testify
C#/.NET	xUnit, NUnit, MSTest, Moq
BDD	Cucumber, SpecFlow, Behave
E2E/Browser	Cypress, Playwright, Selenium
Load	k6, JMeter, Gatling, Locust

15. ✅ Best Practices

- Follow the **pyramid** — many fast unit tests, fewer integration, few E2E.
- One **clear assertion focus** per test; name tests by behavior ("returns_error_when_empty").
- Keep tests **FIRST** (Fast, Isolated, Repeatable, Self-validating, Timely) and **deterministic** (no real time/network/random).
- **Arrange-Act-Assert** structure; avoid logic in tests.
- Test **behavior, not implementation** — don't over-mock; tests shouldn't break on every refactor.
- Fix or quarantine **flaky tests** immediately.
- Run tests in **CI on every PR**; fail fast; track meaningful coverage of critical paths.
- Treat **test code as production code** — review it, refactor it, keep it clean.

16. 🧠 Quick Mental Model

- **Tests = executable spec; catch bugs early, enable safe change.**
- **Pyramid:** lots of **unit** (fast, isolated) → some **integration** (components together) → few **E2E** (whole app, user flows).
- **Unit mocks the DB; integration uses a real test DB; E2E drives the real UI.**
- **TDD** = Red→Green→Refactor (test first, design tool). **BDD** = Given-When-Then in business language (collaboration + living docs).
- **Test doubles:** stub (canned returns) vs mock (verify interactions); fake = lightweight real impl.
- **Coverage shows what ran, not test quality** — aim for meaningful asserts; kill flaky tests.
- Tests are the **gate in CI/CD** — fail fast, shift left, block merges on red.

17. ? Common Interview Questions

Rapid-fire questions interviewers ask about testing:

- **Q: Unit vs integration vs E2E?** — Unit tests one isolated unit (mocked deps). Integration tests components together (real DB/API). E2E tests the whole app through the UI as a user.
- **Q: What is the testing pyramid?** — A guideline: many fast unit tests, fewer integration, few slow E2E — for fast feedback and stability.
- **Q: TDD cycle?** — Red (write a failing test) → Green (minimal code to pass) → Refactor (clean up, stay green).
- **Q: TDD vs BDD?** — TDD tests units in code terms; BDD describes behavior in business language (Given-When-Then) for shared understanding.
- **Q: Mock vs stub?** — A stub returns canned data (controls inputs); a mock has expectations you verify (asserts interactions).
- **Q: Does 100% code coverage mean good tests?** — No — coverage shows what code ran, not that assertions are meaningful. Mutation testing better measures quality.
- **Q: What's a flaky test?** — One that passes/fails nondeterministically; it erodes trust and must be fixed or quarantined.
- **Q: Functional vs non-functional testing?** — Functional = does it do the right thing; non-functional = how well (performance, security, usability).
- **Q: What is regression testing?** — Re-checking existing functionality still works after a change.
- **Q: Smoke vs sanity testing?** — Smoke = quick broad check the build works; sanity = narrow deep check of a specific fix/area.
- **Q: Where do tests fit in CI/CD?** — They're the gate: run on every PR, fail fast (unit→integration→E2E), block merges/deploys on failure.

 Notes compiled as a quick reference for software testing concepts.